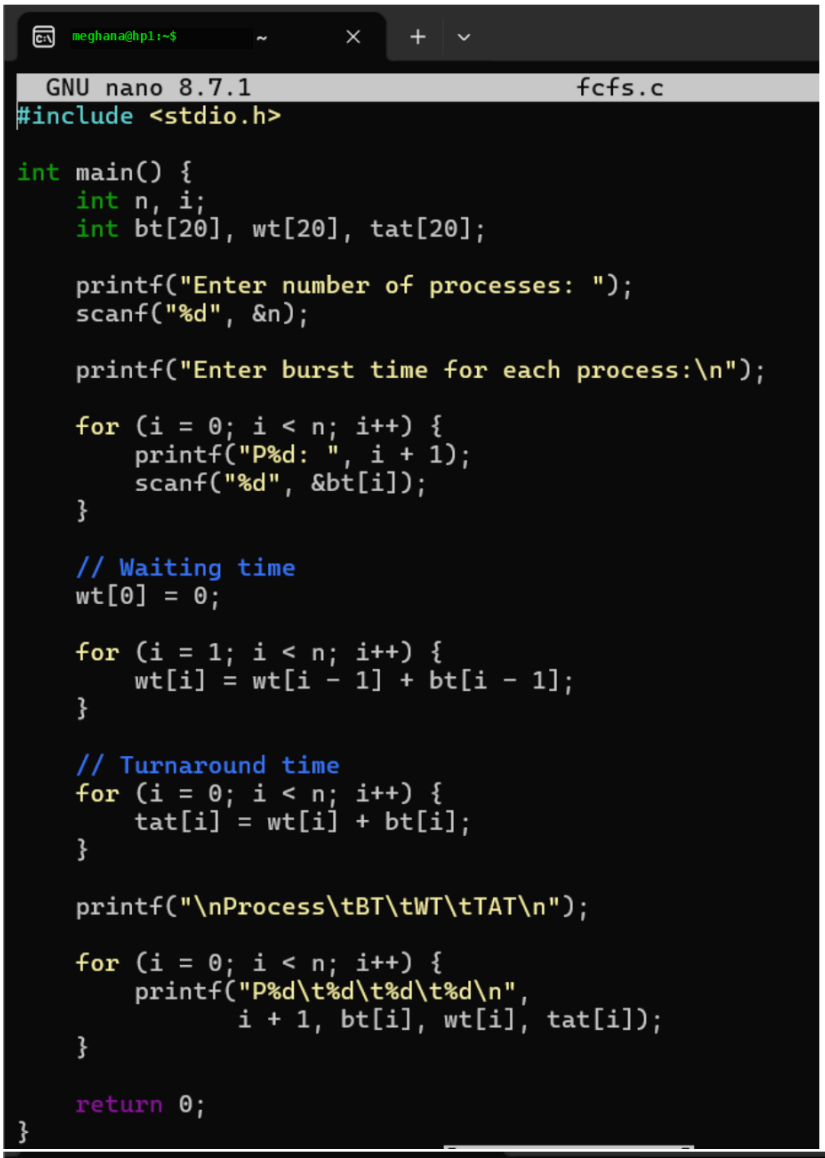
OS PRACTICAL SCHEDULING ALGORITHMS

C.SRI DHRUTHI **2520030490**

1.FCFS Scheduling algorithm



```
meghana@hp1:~$ ~
 GNU nano 8.7.1                                  fcfs.c
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
meghana@hp1:~$ nano fcfs.c
meghana@hp1:~$ gcc fcfs.c -o fcfs
meghana@hp1:~$ ./fcfs
Enter number of processes: 3
Enter burst time for each process:
P1: 5
P2: 3
P3: 8

Process BT      WT      TAT
P1      5       0       5
P2      3       5       8
P3      8       8       16
```

2.SJF_NONpreemptive

```
meghana@hp1:~$ ~
 GNU nano 8.7.1                                  sjf.c *
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort according to burst time
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
```

```
meghana@hp1:~$ nano sjf.c
meghana@hp1:~$ gcc sjf.c -o sjf
meghana@hp1:~$ ./sjf
Enter number of processes: 3
Enter burst time:
P1: 5
P2: 3
P3: 8

Process BT      WT      TAT
P1      3       0       3
P2      5       3       8
P3      8       8       16
meghana@hp1:~$ ~$
```

3.Priority

```
meghana@hp1:~$ ~
 GNU nano 8.7.1                    priority.c *
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    // Smaller number = higher priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {

                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
```

```
meghana@hp1:~$ nano priority.c
meghana@hp1:~$ gcc priority.c -o priority
meghana@hp1:~$ ./priority
Enter number of processes: 3
Enter burst time and priority:
P1 Burst Time: 5
P1 Priority: 2
P2 Burst Time: 3
P2 Priority: 1
P3 Burst Time: 8
P3 Priority: 3

Process BT      Priority        WT      TAT
P1      3       1               0       3
P2      5       2               3       8
P3      8       3               8       16
meghana@hp1:~$
```

4.ROUND ROBIN

```
meghana@hp1:~$ ~
 GNU nano 8.7.1                                round_robin.c *
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin scheduling
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {

            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
```

```
meghana@hp1:~$ nano round_robin.c
meghana@hp1:~$ gcc round_robin.c -o round_robin
meghana@hp1:~$ ./round_robin
Enter number of processes: 3
Enter burst time:
P1: 5
P2: 3
P3: 8
Enter time quantum: 2

Process BT      WT      TAT
P1      5       7       12
P2      3       6       9
P3      8       8       16
```